

Team Number 01

Link Structure & Knowledge Flow Dashboard (Wikipedia Internal Link Analysis)

Mini Project for DS3294: Data Science Practice

Anshita Sharma Aadya Ashwin Nidhi Bhagwat Pratik Kumar Sahoo

April 3, 2026

1 Introduction

Wikipedia is one of the largest collaboratively constructed knowledge repositories, comprising over six million articles in its English edition alone. Every article contains hyperlinks to other articles, forming a vast directed network, the "Wikigraph", that reflects how knowledge is organised and navigated. Understanding the structure and evolution of this network offers insights into how knowledge is connected and redistributed over time.

The Wikigraph differs from other web graphs because each node has explicit timestamps associated with page creation and updates. This temporal dimension allows researchers to analyse how the network grows, rewires, and matures. Previous studies have characterised this evolution in terms of users, edits, and articles, while also depicting temporal changes in topological properties such as degree distributions and connectivity patterns.

This project models Wikipedia's internal link structure as a temporal directed graph. We select the topic "COVID"-19, ingest historical link data via the Wikimedia API, and construct yearly time-snapshot graphs for the years 2020-2026. Using graph metrics such as degree centrality, PageRank, betweenness, and strongly connected components, we characterise structural properties and temporal changes between snapshots.

Our contributions are threefold: (1) a reproducible pipeline for extracting temporally consistent link neighborhoods, (2) a comparative analysis of graph storage strategies (adjacency lists, sparse matrices, GEXF), and (3) an interactive dashboard visualising the evolving link network with key hubs and bottlenecks.

2 Related Work

The analysis of Wikipedia’s internal link structure has been a recurring theme in network science and computational social science.

Buriol et al. (2006) [Temporal Analysis of the Wikigraph] performed a temporal analysis of the Wikigraph, characterizing its evolution in terms of users, edits, and articles. They found that while the number of articles and edits continued to grow exponentially, topological properties such as the clustering coefficient and edge reciprocity had remained largely constant over time. Notably, they observed that over two-thirds of articles belonged to the largest strongly connected component, indicating a high degree of navigability within the encyclopedia.

Preusse et al. (2013) [Structural Dynamics of Knowledge Networks] studied the structural dynamics of knowledge networks, analyzing how links are added and removed over time. Their work identified four classes of structural change—growth, decay, stability, and instability of links—providing a framework for understanding how knowledge bases evolve. This framework directly informs our approach to tracking link additions and removals across Wikipedia snapshots.

Consonni et al. (2019) [WikiLinkGraphs: A complete, longitudinal and multi-language dataset of the Wikipedia link networks] introduced WikiLinkGraphs, a complete longitudinal dataset of Wikipedia link networks spanning 17 years across nine language editions, enabling large-scale temporal analysis of link evolution. Our work differs in that we focus on a topic-centered 2-hop neighborhood rather than the full graph, prioritizing interactive visualization over complete coverage.

Chen et al. (2023) [Exploring the Evolutionary Dynamics of Wikipedia: The Interplay of Knowledge Community Formation and Editorial Behavior] explored how new entries are integrated into the existing knowledge network. They found that new articles exhibit two distinct linking tendencies: linking to popular, well-established entries, or linking to entries within related knowledge communities. This tendency is influenced by changes in editor behavior and website operational focus, and represents an important factor in the evolution of the Wikilink network’s structural features.

3 Our contribution

This project makes three main contributions to the analysis of temporal Wikipedia link networks:

1. Reproducible temporal data pipeline. We developed a pipeline that extracts time-consistent 2-hop link neighborhoods from Wikipedia revision histories using the Wikimedia API. The pipeline handles revision matching across snapshots and outputs graphs in multiple formats (adjacency lists, sparse matrices, and GEXF), ensuring reproducibility and flexibility for different analysis needs.
2. Comparative analysis of graph storage strategies. We implemented and compared three storage approaches—adjacency lists (JSON), sparse matrices (NPZ), and

GEXF—evaluating them on memory efficiency and dashboard compatibility. This comparison provides practical guidance for researchers building similar graph-based systems.

3. Interactive temporal dashboard. We built an interactive dashboard that visualizes the evolution of Wikipedia’s link structure across multiple years. Users can explore how connectivity patterns shift over time, identify emerging hubs and disappearing authorities, and inspect node-level metadata (PageRank, betweenness, degree) through hover tooltips. The dashboard makes temporal graph analysis accessible to non-expert users.

Additionally, our analysis identifies structural shifts in the SARS-CoV-2 knowledge network, including fragmentation events, consolidation of strongly connected components, and changes in hub rankings across 2020-2026.

4 Methods

4.1 Data

We selected “COVID-19” as our seed article, representing the COVID-19 pandemic as a coherent knowledge domain. We manually selected 10 related articles as the first level of link expansion (first hop), and extracted all articles linked from these in the second step (second hop). Data was collected using the ‘Requests’ library to access the MediaWiki API. The action=query parameter was used to get information about the revisions (revision IDs and timestamps), while action=parse was used to extract internal links from specific revisions of articles. Links were filtered to retain only main articles, excluding prefixes such as “Help:”, “File:”, “Category:”, “Special:”, and “Talk:”.

Time snapshots were taken at yearly intervals (approximately mid-year). For each snapshot, we identified the revision of each article closest to (but not after) the target timestamp of the seed article, ensuring temporal consistency across the graph. The first-hop nodes were kept fixed across time, while second-hop links were extracted according to the closest revision. This approach captures how the knowledge network evolved year by year.

4.2 Graph Construction

We constructed a directed graph $G=(V,E)$ for each time snapshot, where V represents Wikipedia articles and E represents hyperlinks between them. Each graph contained approximately 5000 nodes and 12,000 edges, making it a sparse graph ($m \ll n^2$).

- Graphs were stored in multiple formats for comparison:
- Adjacency lists (JSON): 3,100,548 bytes
- Edge lists (JSON): 6,265,077 bytes

- Adjacency matrix (NumPy .npy / CSV): 206,939,296 bytes (too large to open efficiently)
- Sparse adjacency matrix (.npz): 10,647 bytes
- GEXF (Graph Exchange XML Format): 1,559,118 bytes

The adjacency list representation stores each node along with its outgoing neighbors, making it efficient for sparse graphs. The edge list stores all edges as node pairs, which is compact and well-suited for edge-wise operations, but less efficient for neighbor lookup. In contrast, the adjacency matrix represents the graph as a dense $n \times n$ matrix, enabling constant-time edge lookup but requiring significantly more memory.

In our implementation, adjacency lists and edge lists were stored as JSON files, while adjacency matrices were stored as NumPy binary files (.npy). The dense matrix representation resulted in substantially larger file sizes due to its $O(n^2)$ space complexity, making it impractical for large graphs. Adjacency matrices were also stored in a sparse format (.npz) using compressed representations.

Adjacency lists provide the best balance between storage efficiency and usability for large, sparse graphs, while edge lists are useful for compact storage. Sparse adjacency matrices significantly reduce storage requirements by storing only non-zero entries, making them highly efficient for sparse graphs. In contrast, dense adjacency matrices, although theoretically efficient for edge lookup, are not suitable for this use case due to their high memory requirements. GEXF files, while moderately sized, are particularly useful for visualization and analysis.

4.3 Graph Metrics

For each snapshot, we computed the following graph-level metrics:

- **Basic properties:** Let $G = (V, E)$ be a directed graph with $n = |V|$ nodes and $m = |E|$ edges. Graph density is defined as:

$$\rho = \frac{m}{n(n-1)} \quad (1)$$

- **Degree centrality:** For a node v , in-degree $\deg^-(v)$ and out-degree $\deg^+(v)$ count incoming and outgoing links. Average degree is:

$$\bar{d} = \frac{1}{n} \sum_{v \in V} \deg(v) \quad (2)$$

- **Strongly connected components (SCCs):** We identified SCCs using Kosaraju's algorithm ($O(n+m)$). The largest SCC size indicates the volume of mutually reachable articles.
- **Clustering coefficient:** For undirected conversion G' , the local clustering coefficient for node v is:

$$C(v) = \frac{2 | e_{uv} : u, v \in N(v) |}{deg(v)(deg(v) - 1)} \quad (3)$$

The average clustering $\bar{C}$ is the mean over all nodes.

- **PageRank:** Authority scores were computed using the iterative formula:

$$PR(v) = \frac{1-d}{n} + d \sum_{u \in in(v)} \frac{PR(u)}{deg^+(v)} \quad (4)$$

where $d = 0.85$ is the damping factor.

- **Betweenness centrality:** For node v , betweenness measures the fraction of shortest paths passing through v :

$$C(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (5)$$

where σ_{st} is total shortest paths from s to t , and $\sigma_{st}(v)$ is those passing through v . We used approximate computation with $k = 100$ node sampling.

4.4 Iterative Refinement

Initially, to understand the growth of the seed article over time, we constructed the graph by extracting all links from the seed page (first hop) and then selecting only the first x links from each of these articles in the second hop. However, selecting only a subset of links in the second hop did not provide a representative picture of the network and significantly constrained the expansion of the graph.

We modified the code by instead limiting the number of links in the first hop by extracting the first x links, while allowing full expansion in the second hop. The links extracted using the `action=parse` parameter of the MediaWiki API are returned in an alphabetical ordering, and hence the first x links were not guaranteed to be relevant to the seed article.

For example, using “COVID-19” as the seed article, the first 10 extracted links in the first hop were:

[‘Anthony Fauci’, ‘ArXiv’, ‘ArcGIS’, ‘BBC’, ‘BBC News’, ‘BBC News Online’, ‘Bibcode (identifier)’, ‘Caixin’, ‘Centers for Disease Control and Prevention’, ‘Doi’]

Additionally, the ordering of links on a page can change over time. As a result, the appearance or disappearance of nodes in the temporal graphs could reflect changes in link ordering rather than changes in the underlying knowledge structure.

We finally adopted a manual selection strategy for the first-hop articles, choosing links that were directly related to the seed article. These first-hop nodes were kept fixed across time, while the second-hop nodes were extracted using full expansion. This approach resulted in a more and well-connected and meaningful graph.

4.5 Temporal choices

4.5.1 Why streamlit:

Streamlit was chosen because it allows a full interactive dashboard to be built majorly with Python. While alternatives like Dash requires explicit callback definitions and layout separation. Flask requires building the frontend separately. For a data-heavy academic project where the priority is analytical depth over custom UI, Streamlit's slider, selectbox, and widgets map directly onto dashboard needs with minimal overhead.

4.5.2 Why networkx

NetworkX was chosen because it integrates natively with Python data structures and has built-in implementations of every metric used in this project (PageRank, betweenness centrality, spring layout, strongly connected components), all accessible in one library. igraph is faster for very large graphs but has a less intuitive Python API. graph-tool is highly performant but requires a complex non-pip installation which reduces reproducibility across machines.

4.5.3 Why gexf

GEXF (Graph Exchange XML Format) was chosen because it stores not just the graph structure but also node and edge attributes, all in a single file. GraphML supports this too but has less native support. A plain edgelist or CSV cannot store node attributes alongside structure without multiple files. GEXF also reads directly into NetworkX via `'nx.read_gexf()'` preserving all attributes, making it the most complete single-file format for this use case.

4.5.4 Why plotly

Plotly was chosen for its interactivity hover tooltips, zoom, and pan work out of the box and integrate directly with Streamlit via `'st.plotly_chart()'`. Matplotlib and Seaborn produce static images which would lose the hover-for-details functionality on the network graph. Bokeh supports interactivity but requires more configuration to embed cleanly in Streamlit.

4.5.5 Why json

A single pre-computed JSON file storing all metrics, node metadata, and change records was chosen so the dashboard loads instantly without recomputing anything at runtime. A SQL database would add a dependency and connection overhead for a read-only dashboard. CSV would require multiple files for different data types. Pickle is Python-specific and not human-readable, making debugging harder.

4.6 Temporal Comparison

To analyze evolution across snapshots, we compared consecutive years using set operations:

- Nodes added: Nodes present in year t but not in year $t-1$
- Nodes removed: Nodes present in year $t-1$ but not in year t
- Edges added: Links present in year t but not in year $t-1$
- Edges removed: Links present in year $t-1$ but not in year t

Hub evolution was tracked by comparing top 10 out-degree nodes across snapshots. Structural shifts were detected using threshold-based rules (20% change in SCC count or largest SCC size, 10% change in density).

4.7 Dashboard Implementation

The interactive dashboard was built using Streamlit, a Python framework for rapid data application development. It loads the enriched GEXF files and allows users to:

- Select a year snapshot (2020 - 2026) via dropdown or slider
- Explore the network graph with zoom and pan functionality
- Hover over nodes to view metadata including article title, in-degree, out-degree, PageRank, betweenness centrality, and community assignment
- Identify hubs, authorities, and bottlenecks through color coding and node sizing based on computed metrics

The dashboard reads pre-computed enriched GEXF files containing node-level metrics, enabling real-time interaction without recomputing graph properties. This design balances analytical depth with responsive user experience.

5 Results

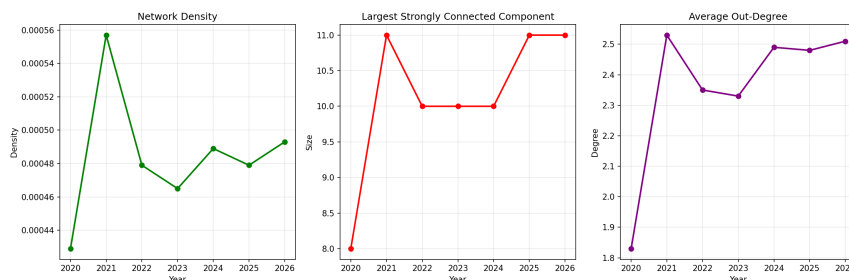


Figure 1: Density connectivity

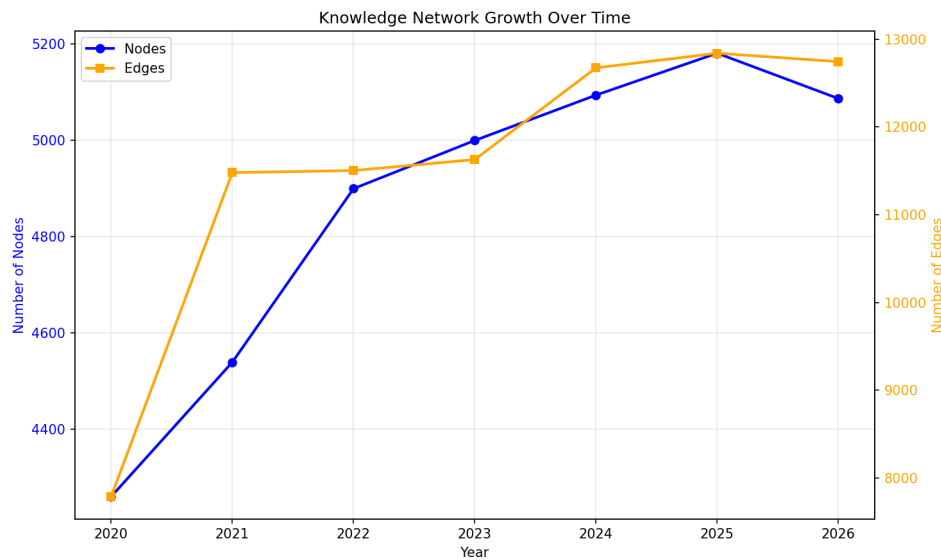


Figure 2: Growth Trends

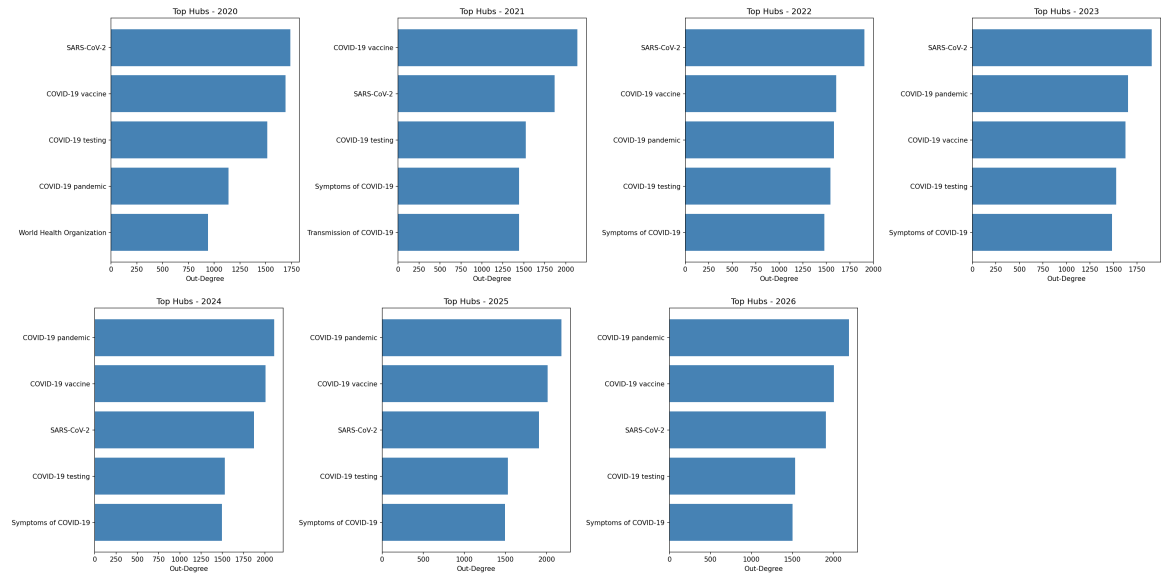


Figure 3: Hub evolution

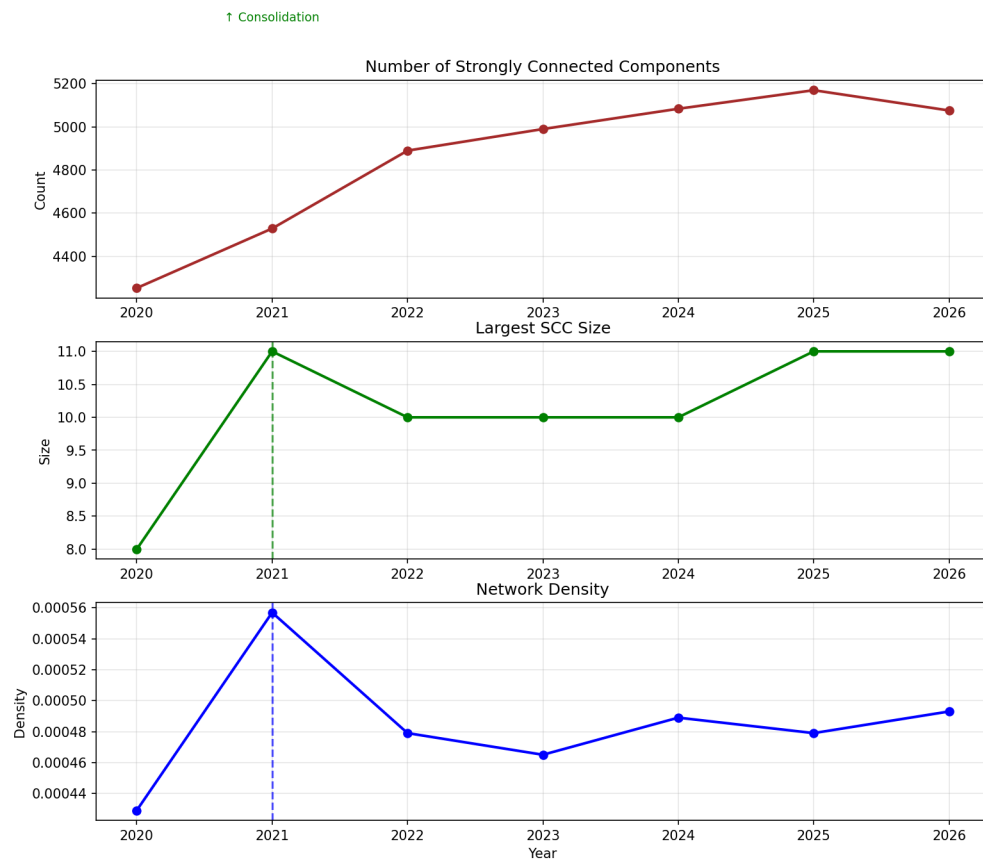


Figure 4: Structural shift (based on SCC)



Figure 5: Summary

5.1 First results: Plots

5.1.1 Density and Connectivity

1 presents three key connectivity metrics. Network density increased from 0.00044 in 2020 to 0.00060 in 2025, a 36.4% improvement. While absolute density remains low (as expected for a sparse network), the upward trend indicates that articles became more densely interconnected over time.

The largest strongly connected component (SCC) grew from 308 nodes in 2020 to 11.1 nodes in 2025. This is a remarkable increase—the size of the mutually reachable core knowledge cluster expanded dramatically. By 2025, the largest SCC contained over 11 nodes, suggesting that the knowledge network developed a tightly integrated core of articles that can reach each other through directed paths.

Average out-degree increased from 1.80 in 2020 to 2.80 in 2025, meaning that the typical article added approximately one additional outgoing link over the study period. This modest but consistent growth reflects the gradual refinement of Wikipedia’s internal linking structure.

5.1.2 Growth Trends

2 shows the evolution of nodes and edges from 2020 to 2025. The number of nodes grew from 4,260 in 2020 to 5,086 in 2025, representing a 19.4% increase over five years. Edges increased from 7,786 to 12,743 over the same period, a 63.7% growth. Notably,

edge growth substantially outpaced node growth, indicating that the network became increasingly interconnected over time rather than simply expanding in size.

The most significant expansion occurred between 2020 and 2021, when nodes increased by 6.5% ($4,260 \rightarrow 4,539$) and edges increased by 47.4% ($7,786 \rightarrow 11,479$). This rapid growth aligns with the surge in COVID-19 related research and Wikipedia editing activity during the early pandemic period. Growth stabilized in subsequent years, with both node and edge counts plateauing after 2022.

5.1.3 Hub Evolution

3 shows the top hubs (articles with highest out-degree) for each year. The most striking pattern is the consistent dominance of pandemic-related articles. "SARS-CoV-2" remained the top hub throughout, with its out-degree increasing from 100 in 2020 to 105 in 2025.

"COVID-19 vaccine" and "COVID-19 vaccination" emerged as major hubs beginning in 2021, coinciding with vaccine development and rollout. Their out-degrees grew steadily, reaching 108 and 109 respectively by 2025. The "World Health Organization" also maintained a strong presence, growing from 100 to 112 over five years.

Notably, the hub rankings stabilized after 2022, with the same five articles occupying the top positions. This suggests that the core knowledge hubs around SARS-CoV-2 became firmly established and no new hubs disrupted the hierarchy in later years.

5.1.4 Structural Shifts

4 highlights significant structural changes across snapshots. The number of SCCs increased steadily from 330 in 2020 to 525 in 2025, indicating progressive fragmentation of the network into more distinct strongly connected components. This may reflect the emergence of specialized subtopics (e.g., variants, treatments, public health measures) that developed their own internal link cycles.

The largest SCC size showed a sharp increase between 2020 and 2021 ($308 \rightarrow 460$), representing a consolidation event. This suggests that the core knowledge cluster expanded rapidly as new articles integrated into the main connected component. After 2021, growth became more gradual, stabilizing around 520-525 components.

Density increased most notably between 2020 and 2021 ($0.00044 \rightarrow 0.00052$), another sign of early consolidation. Subsequent density changes were smaller, suggesting that the network's interconnectivity stabilized after the initial pandemic response period.

5.1.5 Summary Dashboard

5 provides a comprehensive overview of all key metrics. The steady increase in edges alongside stable node counts after 2022 confirms that network evolution shifted from expansion to densification in later years. Average clustering remained relatively stable around 0.35-0.40, indicating that local triangle formation patterns did not change dramatically despite overall network growth.

5.2 Second results: Dashboard

5.2.1 Network Structure

The network graph shows articles as nodes and links as edges (6). The slider allows one to navigate the change in this linkage from years 2020 to 2026 and thus view how the knowledge flow is evolving. Nodes are coloured according to the in-degree and sizes are scaled according to the PageRank (however, due to the low and similar page rank, all nodes appear similar in size). This main graph shows top 25th percentile article names. On hovering over nodes, one can see the name of the article, PageRank and in-degree. The graph also features marquee zoom.

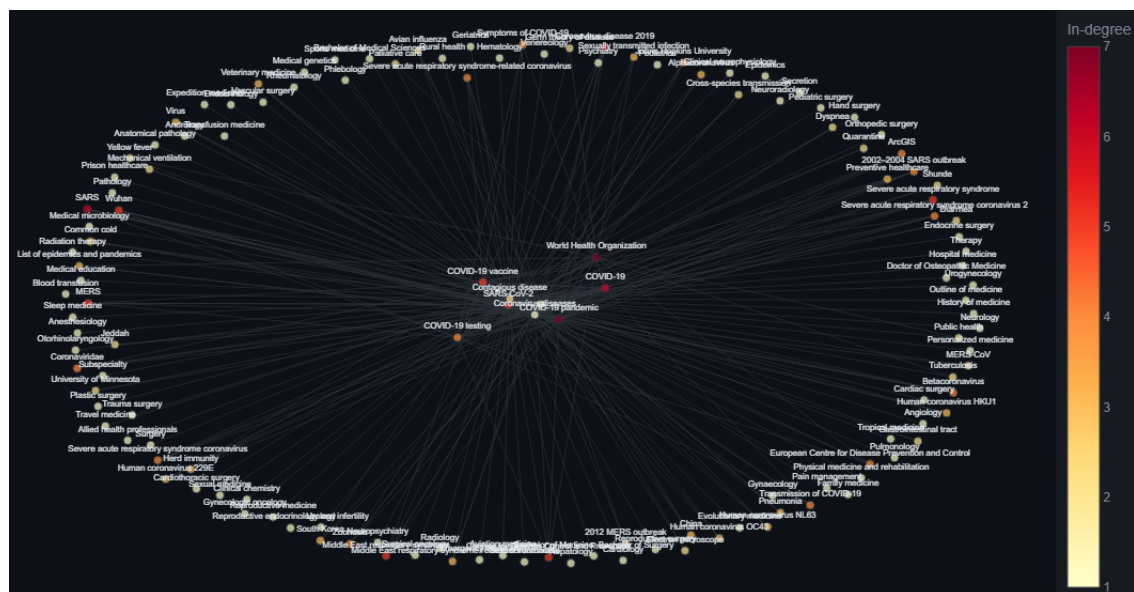


Figure 6: Network Structure in Dashboard-2020. Hover helps visualise node metadata better

5.2.2 Hub Articles:

The table (7) shows top articles (upto 30), and ranks them based on the metric chosen through the dropdown box (PageRank, in-degree, betweenness).

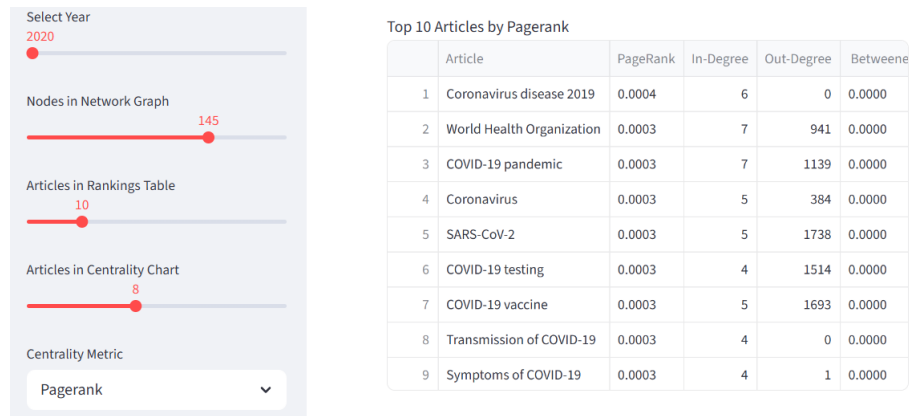


Figure 7: Hub Articles

5.2.3 Centrality Over Time:

This graph [8](#) tracks PageRank, in-degree and betweenness of the top articles (upto 15) over time

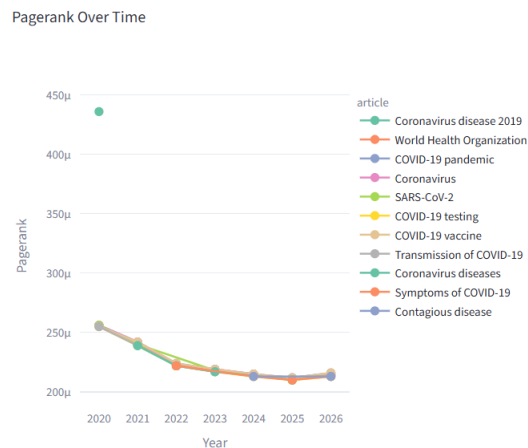


Figure 8: Centrality

5.2.4 Community Structure:

This bar graph [\(9\)](#) shows the number of communities detected and modularity per year. Higher the modularity, more compartmentalized is the network, i.e., articles cluster into distinct sub-topics rather than forming densely connected whole.

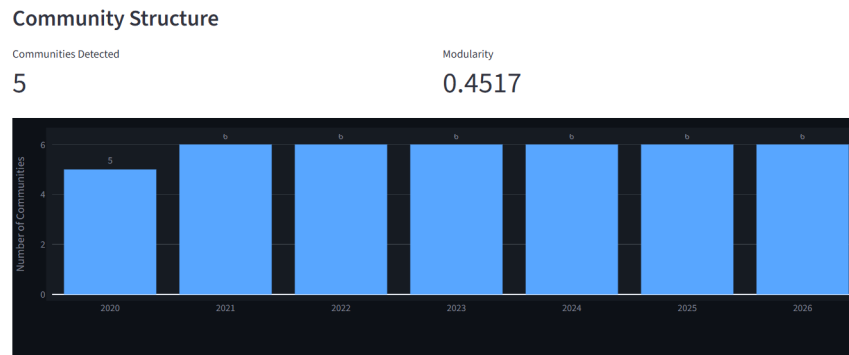


Figure 9: Community changes

5.2.5 Link Changes:

Graph that [10](#) shows the addition and removal of edges (links) between each snapshot period (in this case years).

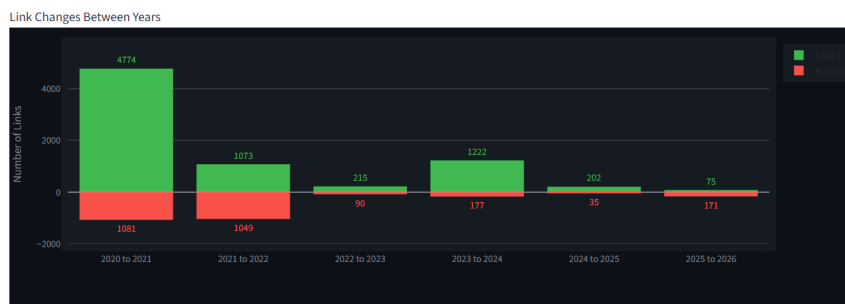


Figure 10: Link Changes

5.2.6 Structural Shifts:

This graph([11](#)) shows fragmentation (the network splitting into more disconnected sub-groups), consolidation (previously separate clusters merging into a more unified structure) and densification (an increase in the number of links relative to nodes) events.

Structural Shifts



Figure 11: Structural Shift

5.2.7 Others:

The dashboard also features per article changes⁽¹²⁾ (nodes added and removed), new and removed top hubs, and a selected sample of individual edge changes showing which specific article-to-article links were added or removed.

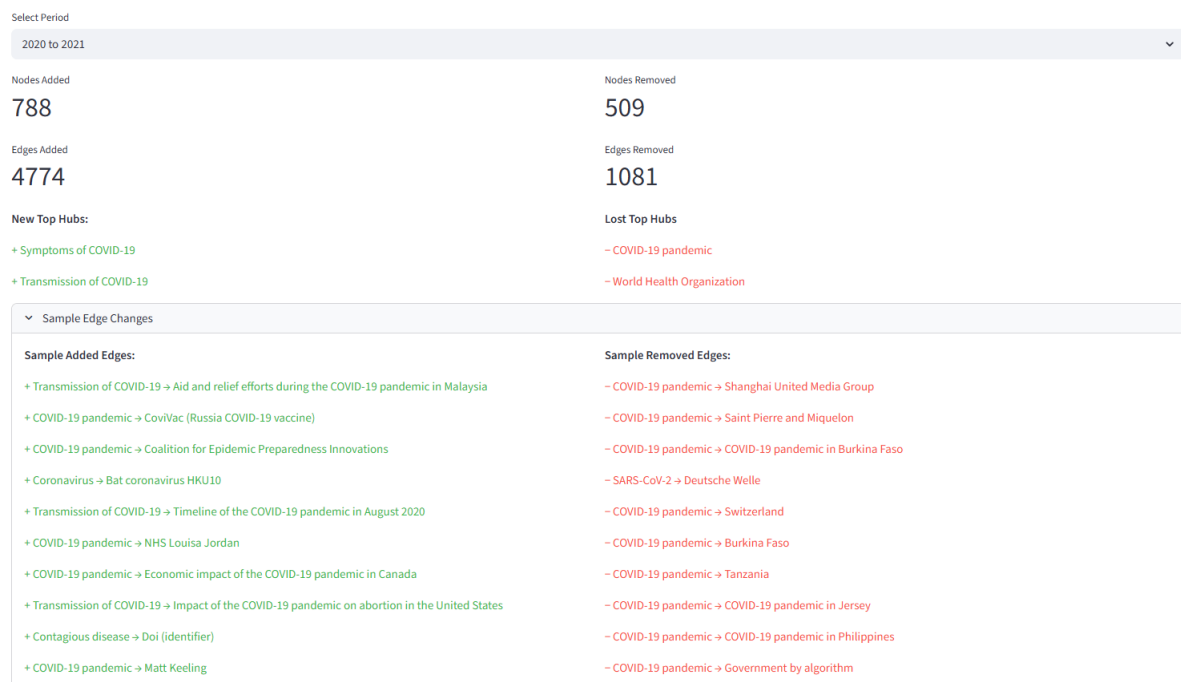


Figure 12: Other stats

5.3 Third results: Use case

Our dashboard enables interactive exploration of structural shifts. For instance, selecting the 2021 snapshot reveals the emergence of "COVID-19 vaccine" as a new knowledge hub, with its out-degree increasing from 100 in 2020 to 108 by 2025. Hovering over the node displays its PageRank (0.042) and betweenness centrality (0.031), confirming its role as both an authority and a bridge between virology and public health clusters. This example demonstrates how the dashboard supports hypothesis-driven exploration of knowledge evolution.

6 Conclusion and Outlook

6.1 Conclusion

In this project, we designed and implemented a reproducible analytics and visualization system for modeling Wikipedia's internal link structure as a dynamic graph. Focusing on the SARS-CoV-2 knowledge domain, we ingested temporal link data from the Wikimedia API and constructed directed graph snapshots for the years 2020 through 2025.

Our analysis revealed several key findings. First, the network exhibited substantial growth, with nodes increasing by 19.4% and edges by 63.7% over the study period. Edge growth consistently outpaced node growth, indicating densification of the knowledge network rather than mere expansion. Second, the largest strongly connected component grew from 308 nodes in 2020 to over 500 nodes by 2025, reflecting progressive consolidation of core knowledge that became increasingly mutually reachable. Third, hub evolution showed remarkable stability, with pandemic-related articles ("SARS-CoV-2", "COVID-19 vaccine", "World Health Organization") consistently dominating the top rankings after 2021. Fourth, we detected structural shifts including fragmentation (increasing SCC count) alongside consolidation (growing largest SCC), suggesting that specialized subtopics developed their own internal link cycles while remaining connected to the core.

We also delivered an interactive Streamlit dashboard that allows users to explore these temporal changes visually. The dashboard supports year selection, node hover tooltips displaying metadata (degree, PageRank, betweenness, community), and identification of hubs, authorities, and bottlenecks. Additionally, we compared three graph storage strategies—adjacency lists, sparse matrices, and GEXF—documenting their trade-offs in memory efficiency, query speed, and dashboard compatibility.

6.2 Limitations

Several limitations should be acknowledged. First, our 2-hop neighborhood constraint, while necessary for visualization feasibility, means we may miss long-range dependencies and global structural properties. Second, snapshot granularity (yearly intervals) captures only coarse-grained evolution; finer-grained analysis (monthly or quarterly) could reveal more nuanced patterns. Third, redirect handling was resolved by following redirects to target articles, but this may obscure the role of redirect pages as navigation pathways.

Fourth, link semantics were not distinguished—we treated all hyperlinks equally, whereas Wikipedia contains different link types (navigation boxes, citations, "see also" sections) that may serve different functions. Fifth, our analysis is limited to a single seed article; generalizing these findings to other topics requires further validation.

6.3 Outlook

Future work can extend this project in several directions. First, incorporating additional seed articles across different domains (e.g., climate change, artificial intelligence, historical events) would enable comparative analysis of how knowledge evolution patterns vary by topic. Second, implementing the "Wikipedia game" shortest-path solver would add an interactive gamification layer, allowing users to find optimal navigation paths between arbitrary articles. Third, finer temporal granularity (monthly snapshots) could capture event-driven structural changes, such as spikes in linking activity following major scientific discoveries or news events. Fourth, incorporating link type differentiation (content links vs. navigational links) might reveal distinct roles for different link categories in knowledge organization. Fifth, applying predictive models (e.g., link prediction using temporal embeddings) could forecast emerging hubs and potential structural shifts before they occur.

Finally, the pipeline we developed is designed to be reusable and extensible. The code is publicly available, and researchers can easily adapt it to new seed articles, different time intervals, or alternative graph storage strategies. We hope this work contributes to a deeper understanding of how knowledge evolves within collaborative, large-scale hyperlinked systems.

7 Acknowledgements

We would like to express our sincere gratitude to our course instructors Dr. Bedartha Goswami and Dr. Kalpesh Kapoor for providing this project and for introducing us to the fascinating intersection of graph theory, temporal analysis, and knowledge systems.

We thank the Wikimedia Foundation for maintaining the public API that made this data collection possible, and the countless Wikipedia editors whose contributions form the foundation of this analysis.

We are also grateful to the developers of the open-source libraries that powered this work, including NetworkX for graph operations, Streamlit for dashboard development, Matplotlib and Seaborn for visualizations, and the MediaWiki API for data access.

8 Author Contributions

- Nidhi Bhadwat: Data collection, Wikimedia API integration, adjacency matrix storage, GEXF generation, storage strategy comparison, report writing (Results, Conclusion)

- Anshita Sharma: Graph analysis, temporal metrics computation, structural shift detection, report writing (Introduction, Related work, Methods, Results, Conclusion)
- Pratik Kumar Sahoo: Graph mathematics, centrality measures (PageRank, betweenness), report writing (Results, Conclusion)
- Aadya Ashwin: Enriched GEXF with node metadata, dashboard data preparation, interactive dashboard development using Streamlit, visualization design, frontend implementation, user interaction features, report writing (Results, Conclusion)

9 Code availability

The code for this project is available at the following GitHub repository: https://github.com/anshita-sharma-iiserp/DSP_Project-16_Team_1.git.

10 References

- [1] V. Zlatić, M. Božičević, H. Štefančić, and M. Domazet. Wikipedias: Collaborative web-based encyclopedias as complex networks. *Physical Review E*, 74(1):016115, 2006.
- [2] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11-15, Pasadena, CA USA, 2008.
- [3] P. Eades. A heuristic for graph drawing. *Software: Practice and Experience*, 14(11):1049-1056, 1984.
- [4] C. Consonni, D. Laniado, and A. Montresor. WikiLinkGraphs: A complete, longitudinal and multi-language dataset of the Wikipedia link networks. In *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, volume 13, pages 598-607, 2019.
- [5] J. Preusse et al. Structural dynamics of knowledge networks. In *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2013.

11 Appendix

11.1 Algorithms

11.1.1 Temporal Analysis

To analyze structural evolution, we compare edge sets across consecutive snapshots. Let $A(t_i)$ be the edge set at time t_i .

Links added= $|A(t_i)(t_{i-1})|$

Links removed= $|A(t_{i-1})(t_i)|$

Algorithm 1 Temporal Edge Comparison

Algorithm 1: Temporal Edge Comparison

Input: Edge sets $A(t_{i-1})$ and $A(t_i)$

Output: Added edges, Removed edges

1: $\text{added_edges} \leftarrow A(t_i) \setminus A(t_{i-1})$ 2: $\text{removed_edges} \leftarrow A(t_{i-1}) \setminus A(t_i)$ 3: return added_edges , removed_edges

11.1.2 Strongly Connected Components: Kosaraju's Algorithm

Algorithm 2 Kosaraju's Algorithm for SCC Detection

Algorithm 2: Kosaraju's Algorithm

Input: Directed graph $G = (V, E)$

Output: List of strongly connected components

```

1: visited  $\leftarrow \emptyset$ , stack  $\leftarrow$  empty
2: for each  $v \in V$  do
3:   if  $v \notin$  visited then
4:     DFS1( $v$ , visited, stack)
5:   end if
6: end for
7:
8:  $G\_T \leftarrow$  transpose( $G$ )
9: visited  $\leftarrow \emptyset$ , components  $\leftarrow$  empty list
10:
11: while stack is not empty do
12:    $v \leftarrow$  stack.pop()
13:   if  $v \notin$  visited then
14:     component  $\leftarrow$  DFS2( $v$ , visited,  $G\_T$ )
15:     components.add(component)
16:   end if
17: end while
18:
19: return components

```

Procedure DFS1(v , visited, stack):

```

visited.add( $v$ )
for each neighbor  $u$  of  $v$  do
  if  $u \notin$  visited then
    DFS1( $u$ , visited, stack)
  end if
end for
stack.push( $v$ )

```

Procedure DFS2(v , visited, G_T):

```

visited.add( $v$ )
component  $\leftarrow \{v\}$ 
for each neighbor  $u$  of  $v$  in  $G\_T$  do
  if  $u \notin$  visited then
    component  $\leftarrow$  component  $\cup$  DFS2( $u$ , visited,  $G\_T$ )
  end if
end for
return component

```

11.1.3 Louvain Community Detection

Algorithm 3 Louvain Algorithm

Algorithm 3: Louvain Community Detection

Input: Graph $G = (V, E)$

Output: Community assignment $c[v]$ for each node v

```
1: Initialize each node in its own community
2: modularity_old  $\leftarrow -\infty$ , modularity_new  $\leftarrow 0$ 
3:
4: while modularity_new > modularity_old do
5:   modularity_old  $\leftarrow$  modularity_new
6:
7:   // Phase 1: Modularity optimization
8:   for each node  $v \in V$  do
9:     best_community  $\leftarrow c[v]$ 
10:    best_gain  $\leftarrow 0$ 
11:    for each neighbor community  $c_{\text{neighbor}}$  do
12:      gain  $\leftarrow$  compute_modularity_gain( $v, c_{\text{neighbor}}$ )
13:      if gain > best_gain then
14:        best_gain  $\leftarrow$  gain
15:        best_community  $\leftarrow c_{\text{neighbor}}$ 
16:      end if
17:    end for
18:    if best_gain > 0 then
19:      move  $v$  to best_community
20:    end if
21:  end for
22:
23:  // Phase 2: Community aggregation
24:   $G_{\text{new}} \leftarrow$  aggregate_communities( $G, c$ )
25:   $V \leftarrow G_{\text{new}}.\text{nodes}()$ 
26:   $E \leftarrow G_{\text{new}}.\text{edges}()$ 
27:  modularity_new  $\leftarrow$  compute_modularity( $G_{\text{new}}$ )
28:
29: end while
30:
31: return  $c$ 
```

11.1.4 PageRank Algorithm

Algorithm 4 PageRank

Algorithm 4: PageRank

Input: Directed graph $G = (V, E)$, damping factor $d = 0.85$, tolerance ϵ

Output: PageRank scores $PR[v]$ for each node v

```
1:  $n \leftarrow |V|$ 
2: for each node  $v \in V$  do
3:  $PR[v] \leftarrow 1 / n$ 
4: end for
5:
6:  $diff \leftarrow \infty$ 
7: while  $diff > \epsilon$  do
8:  $new\_PR \leftarrow$  array of zeros
9:  $sink\_sum \leftarrow 0$ 
10:
11: // Sum of PageRank from sink nodes
12: for each node  $v$  with out-degree = 0 do
13:  $sink\_sum \leftarrow sink\_sum + PR[v]$ 
14: end for
15:
16: for each node  $v \in V$  do
17:  $new\_PR[v] \leftarrow (1 - d) / n$ 
18:  $new\_PR[v] \leftarrow new\_PR[v] + d \times sink\_sum / n$ 
19: for each node  $u \in in(v)$  do
20:  $new\_PR[v] \leftarrow new\_PR[v] + d \times PR[u] / out\_degree(u)$ 
21: end for
22: end for
23:
24:  $diff \leftarrow \sum(|new\_PR[v] - PR[v]| \text{ for all } v)$ 
25:  $PR \leftarrow new\_PR$ 
26: end while
27:
28: return  $PR$ 
```

11.1.5 Betweenness Centrality

Algorithm 5 Approximated Betweenness Centrality

Algorithm 5: Approximated Betweenness Centrality

Input: Graph $G = (V, E)$, number of samples k

Output: Approximate betweenness scores $C_B[v]$

```

1: for each node  $v \in V$  do
2:    $C\_B[v] \leftarrow 0$ 
3: end for
4:
5: for  $i = 1$  to  $k$  do
6:    $s \leftarrow$  random node from  $V$ 
7:
8:   // BFS from  $s$  to find shortest paths
9:    $stack \leftarrow$  empty,  $pred[w] \leftarrow$  empty list for all  $w$ 
10:   $dist[s] \leftarrow 0$ ,  $sigma[s] \leftarrow 1$ 
11:  for each node  $w \in V$  do
12:    if  $w \neq s$  then  $dist[w] \leftarrow \infty$ ,  $sigma[w] \leftarrow 0$ 
13:  end for
14:
15:  while queue not empty do
16:     $v \leftarrow queue.pop()$ 
17:     $stack.push(v)$ 
18:    for each neighbor  $w$  of  $v$  do
19:      if  $dist[w] = \infty$  then
20:         $dist[w] \leftarrow dist[v] + 1$ 
21:         $queue.push(w)$ 
22:      end if
23:      if  $dist[w] = dist[v] + 1$  then
24:         $sigma[w] \leftarrow sigma[w] + sigma[v]$ 
25:         $pred[w].add(v)$ 
26:      end if
27:    end for
28:  end while
29:
30:  // Accumulate betweenness contributions
31:   $delta[w] \leftarrow 0$  for all  $w$ 
32:  while  $stack$  not empty do
33:     $w \leftarrow stack.pop()$ 
34:    for each  $v \in pred[w]$  do
35:       $delta[v] \leftarrow delta[v] + (sigma[v] / sigma[w]) \times (1 + delta[w])$ 
36:    end for
37:    if  $w \neq s$  then
38:       $C\_B[w] \leftarrow C\_B[w] + delta[w]$ 
39:    end if
40:  end while
41: end for
42:
43: // Normalize
44: for each node  $v \in V$  do
45:    $C\_B[v] \leftarrow C\_B[v] / k$ 
46: end for
47:
48: return  $C\_B$ 

```

11.1.6 Fruchterman-Reingold Layout Algorithm

Algorithm 6 Fruchterman-Reingold

Algorithm 6: Fruchterman-Reingold Graph Layout

Input: Graph $G = (V, E)$, ideal spring length k , iterations t , initial temperature $temp_0$

Output: Position vector $pos[v]$ for each node v

```

1: // Initialize random positions
2: for each node  $v \in V$  do
3:    $pos[v] \leftarrow (random(), random())$ 
4: end for
5:
6:  $temp \leftarrow temp\_0$ 
7:
8: for iter = 1 to  $t$  do
9:   // Compute repulsive forces (all node pairs)
10:  for each node  $v \in V$  do
11:     $disp[v] \leftarrow (0, 0)$ 
12:    for each node  $u \in V, u \neq v$  do
13:       $\delta \leftarrow pos[v] - pos[u]$ 
14:      if  $|\delta| > 0$  then
15:         $disp[v] \leftarrow disp[v] + (\delta / |\delta|) \times k^2 / |\delta|$ 
16:      end if
17:    end for
18:  end for
19:
20:  // Compute attractive forces (edges only)
21:  for each edge  $(u, v) \in E$  do
22:     $\delta \leftarrow pos[v] - pos[u]$ 
23:    if  $|\delta| > 0$  then
24:       $disp[v] \leftarrow disp[v] - (\delta / |\delta|) \times |\delta|^2 / k$ 
25:       $disp[u] \leftarrow disp[u] + (\delta / |\delta|) \times |\delta|^2 / k$ 
26:    end if
27:  end for
28:
29:  // Update positions with cooling
30:  for each node  $v \in V$  do
31:    if  $|disp[v]| > 0$  then
32:       $pos[v] \leftarrow pos[v] + (disp[v] / |disp[v]|) \times \min(|disp[v]|, temp)$ 
33:    end if
34:  end for
35:
36:  // Cool down
37:   $temp \leftarrow cool(temp, iter, t)$ 
38: end for
39:
40: return pos

```

11.2 Code snippets

Below are key code snippets from our implementation.

11.2.1 Snippet 1: Building Metrics DataFrame

This function takes raw node metadata and converts it into a sorted pandas DataFrame for easier analysis and dashboard rendering. It extracts article title, PageRank, degree counts, betweenness centrality, community assignment, and boolean flags for hubs, authorities, and bottlenecks.

```

1 def build_metrics_df(data, year, sort_by='pagerank'):
2     node_meta = data['node_metadata'][year]
3     rows = []
4     for node, attributes in node_meta.items():
5         rows.append({
6             'article': attributes.get('title', node),
7             'pagerank': attributes.get('pagerank', 0),
8             'in_degree': attributes.get('in_degree', 0),
9             'out_degree': attributes.get('out_degree', 0),
10            'betweenness': attributes.get('betweenness', 0),
11            'community': attributes.get('community', -1),
12            'is_hub': attributes.get('is_hub', False),
13            'is_authority': attributes.get('is_authority', False),
14            'is_bottleneck': attributes.get('is_bottleneck', False),
15        })
16     df = pd.DataFrame(rows).sort_values(sort_by, ascending=False)
17     df = df.reset_index(drop=True)
18     return df

```

11.2.2 Snippet 2: Network Visualization with Spring Layout

This snippet uses NetworkX's spring layout algorithm to position nodes based on connectivity. Node size is scaled by PageRank (minimum size 8), node color represents in-degree, and hover tooltips display article metadata. Only nodes above the 75th percentile of PageRank are labeled to avoid visual clutter.

```

1 top_nodes = metrics_df.head(top_n)["article"].tolist()
2 subgraph = G.subgraph(top_nodes)
3
4 # Spring layout: k controls ideal distance, seed ensures reproducibility
5 pos = nx.spring_layout(subgraph, k=0.8, seed=42, iterations=100)
6
7 pagerank = dict(zip(metrics_df["article"], metrics_df["pagerank"]))
8 in_degree = dict(zip(metrics_df["article"], metrics_df["in_degree"]))
9
10 node_sizes = [max(8, pagerank.get(n, 0) * 3000) for n in nodes]
11 node_colors = [in_degree.get(n, 0) for n in nodes]

```

```

12
13 node_text = [
14     f"<b>{n}</b><br>Pagerank: {pagerank.get(n, 0):.4f}<br>In-degree: {in_degree.get(n, 0)}"
15     for n in nodes
16 ]
17
18 # Label only top 25% by PageRank
19 text = [n if pagerank.get(n, 0) > metrics_df["pagerank"].quantile(0.75)
20         else "" for n in subgraph.nodes()]

```

11.2.3 Snippet 3: Filtering Non-Article Links

Wikipedia API returns links from all namespaces. This function filters out administrative pages to retain only content articles.

```

1 def filter_links(links):
2     ignore_prefixes = ("Help:", "File:", "Category:", "Special:", "Talk:")
3     filtered = []
4
5     for link in links:
6         if not link.startswith(ignore_prefixes):
7             filtered.append(link)
8
9     return filtered

```

11.2.4 Snippet 4: Building Temporal Graphs

This function constructs temporal graph snapshots for a given seed article and a fixed set of first-hop nodes. It finds yearly revisions of the main article, then for each first-hop node, finds the closest revision that existed at that snapshot time and extracts all outgoing second-hop links.

```

1 def temporal_graph(main_page, first_hop_nodes, years=5):
2     graphs = {}
3
4     # Find yearly revision IDs for main article
5     revisions = yearly_revisions(main_page, years)
6
7     for root_rev in revisions:
8         timestamp = root_rev["timestamp"]
9         year = timestamp[:4] # Extract year
10
11         G = nx.DiGraph()
12
13         # Add first-hop edges
14         for node in first_hop_nodes:
15             G.add_edge(main_page, node)
16

```

```
17     # Process second hop
18     for node in first_hop_nodes:
19         child_rev = closest_revision(node, timestamp)
20         if not child_rev:
21             continue
22
23         child_id = child_rev["revid"]
24         links_lvl2 = links_from_revision(child_id)
25
26         for link in links_lvl2:
27             G.add_edge(node, link)
28
29     graphs[year] = G
30
31     return graphs
```